

UNIT-1

1. INTRODUCTION to Database Systems

- **Introduction**
 - What is a database management system?
 - Why study databases? Why not use file systems?
 - The three-level architecture
 - Schemas and instances
- **Overview**
 - Data models, E-R model, Relational model
 - Data Definition Language, Data Manipulation Language
 - SQL
 - Transaction Management, Storage Management
 - User types, database administrator
 - System Structure

1.1 What is a Database Management System?

- **Collection of data**
 - Interrelated data
 - Relevant to some endeavour
- **Software to access the data**
 - Convenient
 - Efficient
- **History**
 - 1950s-60s: magnetic tape and punched cards
 - 1960s-70s: hard disks, random access, file systems
 - 1970s-80s: relational model becoming competitive
 - 1980s-90s: relational model dominant, object-oriented databases
 - 1990s-00s: web databases and XML

Why Study Databases?

- They touch every aspect of our lives
- **Applications:**
 - Banking: all transactions
 - Airlines: reservations, schedules
 - Universities: registration, course enrolment, grades
 - Sales: customers, products, purchases
 - Manufacturing: production, inventory, orders, supply chain
 - Human resources: employee records, salaries, tax deductions
 - Telecommunications: subscribers, usage, routing
 - Computer accounts: privileges, quotas, usage
 - Records: climate, stock market, library holdings
- **Explosion of unstructured data on the web:**
 - Large document collections

- Image databases, streaming media

1.2 Why not use file systems?

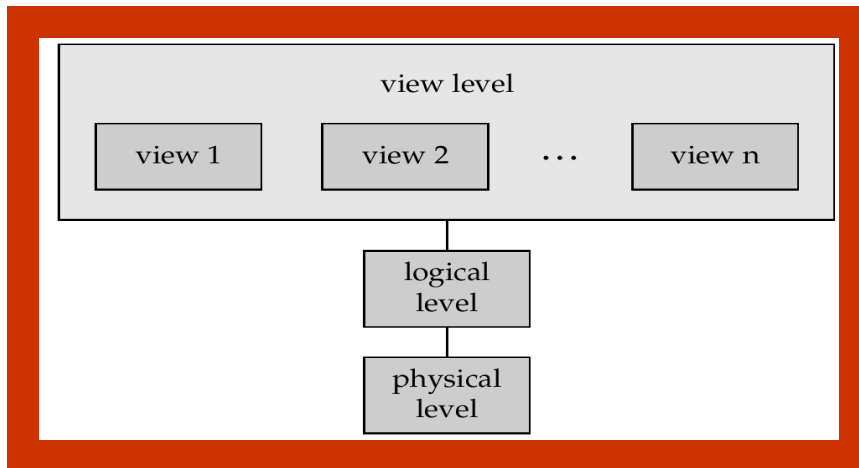
- **Data redundancy and inconsistency**
 - Multiple file formats
 - Duplication of information in different files
- **Difficulty in accessing data**
 - Need to write a new program to carry out each new task
- **Data isolation**
 - Multiple files and formats
- **Integrity problems**
 - Integrity constraints (e.g. account balance > 0) become part of program code
 - Hard to add new constraints or change existing ones
- **Maintenance problems**
 - When we add a new field, all existing applications must be modified to ignore it
- **Atomicity of updates**
 - Failures may leave database in an inconsistent state with partial updates carried out
 - E.g. transfer of funds from one account to another should either complete or not happen at all
- **Concurrent access by multiple users**
 - Concurrent accessed needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - E.g. two people reading a balance and updating it at the same time
 - Security problems
- **Database systems offer solutions to all the above problems**

1.3 The Levels of Abstraction

- **Physical level:** how a record is stored on disk
- **Logical level:** describes data stored in database, and the relationships among the data.
- type customer = record


```

      name : string;
      street : string;
      city : integer;
      end;
```
- **View level:** application-specific selections and arrangements of the data
- hide details of data types
- Views can also hide information for security reasons



1.4 Schemas vs Instances

- **Schema**
 - the logical structure of the database.
 - e.g., the database consists of information about a set of customers and accounts and the relationship between them.
 - Analogous to type information of a variable in a program.
- **Instance**
 - the actual content of the database at a particular point in time.
 - Analogous to the value of a variable.
- **Data Independence**
 - the ability to modify a schema in one-level (i.e. Internal Schema or Conceptual Schema) without affecting a schema in the next-higher-level (i.e. Conceptual or External schema)
 - Applications depend on the logical schema
 - Database engines take care of efficient storage and query processing
 - *Data independence are of two types:*
 - Physical Data Independence: Physical data independence is the ability to modify the physical schema – i.e. internal schema which describes the physical storage devices or structure to store the data – without affecting the conceptual schema – application programs.
 - Logical Data Independence: Logical data is the ability to modify the logical schema – i.e. Conceptual Schema, which decides what information is to be kept in the database – without affecting the next higher level schema – i.e., External Schema – application program.

1.5 Data Models

- A collection of tools for describing
 - data
 - data relationships
 - data semantics
 - data constraints
- The data models are divided into the three different groups
 - Object-Based Logical Data Models
 - Record-Based Logical Data Models

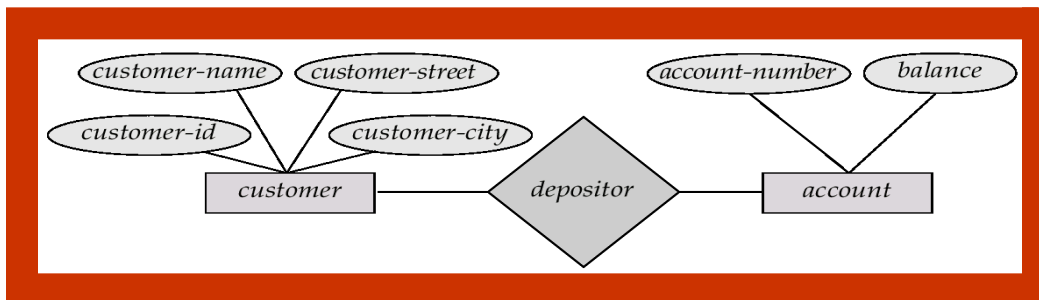
- Physical Data Models

□ Object-Based Logical Model

- Object – based logical models are used in describing data at logical level and view level. Logical and view levels are used to retrieve the data.
- *Object-Based Logical Models are described in the different following models:*
 - The Entity-Relationship Model
 - Object-Oriented Model
 - The Semantic Data Model
 - The Functional Data Model

■ **Entity-Relationship Model**

- An entity is a thing or object in the real world that is distinguishable from other objects.
- The Entity – Relationship Model is based on a collection of basic objects, called entities, and the relationship among these objects.
- Example of schema in the entity-relationship model



- ◆ Rectangles represent entities
- ◆ Diamonds represent relationship among entities
- ◆ Ellipse represents attributes
- ◆ Lines represents links of attributes to entities to relations

■ **Object-Oriented Model**

- Like the E-R model, the Object-Oriented Model is based on a collection of objects. An object contains values stored in instance variables, within the object also contains bodies of code that operates on the object. These bodies of code are called as methods.
- Objects that contain the same types of values and the same methods are grouped together into classes. A class may be viewed as a definition for objects. This combination of data and methods comprising a definition is similar to a Programming-language abstract data type.
- The only way in which one object can access the data of another object is by invoking a method of that other object. This action is called sending a message to the object.

■ **The Semantic Data Model**

- A semantic data model is a more high-level data model makes it easier for a user to give starting description of data in an enterprise.
- These models contain a wide variety of relations that helps to describe a real application scenario.
- A DBMS cannot support all these relation directly; so it is built only with few relations known as relational model in DBMS.
- A widely used semantic data model is the E-R data model which allows us to graphically denote entities and relationship between them.

■ ***The Functional Data Model***

- The functional Data Model makes it easier to define functions and call them wherever necessary to process data.

□ **Record-Based Logical Models**

- Describes data at logical and view levels.
- Compared with object-based data models, the record-based logical models specify the overall logical structure of the database and provide higher level implementation.
- *The Record – Based models are of three types*
 - Relational Model
 - Network Model
 - Hierarchical Model

■ **Relational Model**

- The relational model represents both data (entities) and relationships among in the form of tables. Each table has multiple columns and each column has a unique name.

■ **Example of tabular data in the relational model**



<i>Customer-id</i>	<i>customer-name</i>	<i>customer-street</i>	<i>customer-city</i>	<i>account-number</i>
192-83-7465	Johnson	Alma	Palo Alto	A-101
019-28-3746	Smith	North	Rye	A-215
192-83-7465	Johnson	Alma	Palo Alto	A-201
321-12-3123	Jones	Main	Harrison	A-217
019-28-3746	Smith	North	Rye	A-201

- **A Sample Relational Database**

customer-id	customer-name	customer-street	customer-city
192-83-7465	Johnson	12 Alma St.	Palo Alto
019-28-3746	Smith	4 North St.	Rye
677-89-9011	Hayes	3 Main St.	Harrison
182-73-6091	Turner	123 Putnam Ave.	Stamford
321-12-3123	Jones	100 Main St.	Harrison
336-66-9999	Lindsay	175 Park Ave.	Pittsfield
019-28-3746	Smith	72 North St.	Rye

(a) The customer table

account-number	balance
A-101	500
A-215	700
A-102	400
A-305	350
A-201	900
A-217	750
A-222	700

(b) The account table

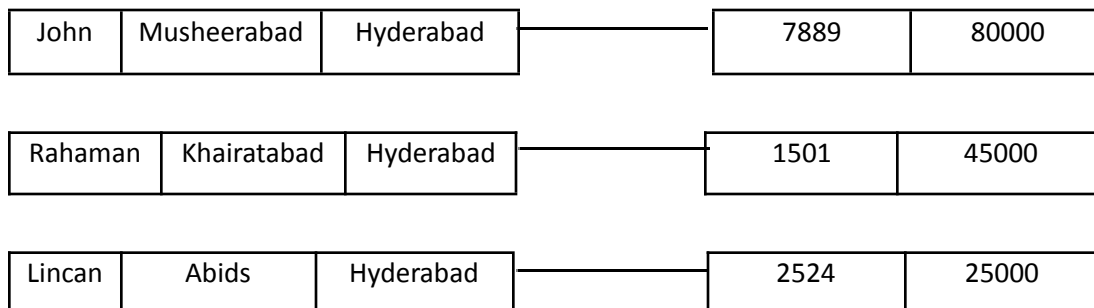
customer-id	account-number
192-83-7465	A-101
192-83-7465	A-201
019-28-3746	A-215
677-89-9011	A-102
182-73-6091	A-305
321-12-3123	A-217
336-66-9999	A-222
019-28-3746	A-201

(c) The depositor table

- The description of data in terms of tables is called as relations, from the above Customer and Accounts relations, we can make a condition that customer details are maintained in Customer table and their deposit details are maintained in the account table database.

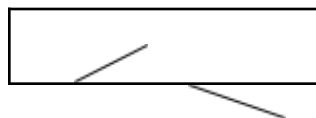
■ Network Model

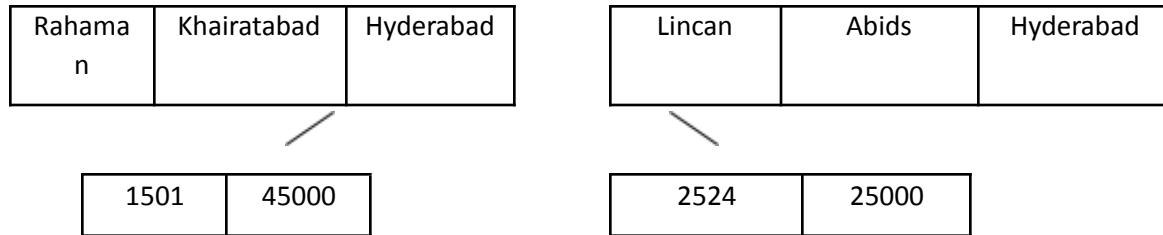
- Data in the network model are represented by collection of records and relationships among data are connected by Links.
- The links can be viewed as pointers..
- The records in the database are represented in the form of graphs.



■ Hierarchical Model

- Hierarchical model is same as the network model. i.e., Data in the Hierarchical model also are represented by collection of records and relationship among data is connected by links.
- The links can be viewed as pointers.
- But, the difference from Network Model is that, the records in the database are represented in the form of trees.





□ Physical Data Models

- Physical data models are used to describe data at the lowest level, which explains how the data is actually stored using complex low-level data structures.
- Actually, the physical data models are rarely used.
- *Two types of Physical Data Models are:*
 - Unifying Model
 - Frame-Memory Model

1.6 DATABASE LANGUAGES

- A DBMS provides two types of Languages:
 - Data-Definition Language (DDL) – to define the data in the database.
 - Data-Manipulation Language (DML) – to manipulate the data in the database.

■ Data Definition Language (DDL)

- Specification notation for defining the database schema
 - E.g.


```
create table account (
    account-number char(10),
    balance integer)
```
- DDL compiler generates a set of tables stored in a *data dictionary*:
 - Database schema
 - Specification of storage structures and access methods

■ Data Manipulation Language (DML)

- Language for accessing and manipulating the data organized by the appropriate data model
 - DML also known as query language
- Two classes of languages
 - Procedural – user specifies what data is required and how to get those data

- Nonprocedural – user specifies what data is required without specifying how to get those data
- SQL is the most widely used query language

■ Data Sub-Language

- The DDL and DML are collectively referred to as the data sub-language when embedded within a host language (Ex: C or COBOL etc.)

■ Host Language

- SQL and QUEL only provide facilities to define and retrieve data. To extend the data manipulation operations of these languages, for example to separately process each type of relation, these languages have to be used with traditional high-level language. Such a language is called a *host language* and the program is called the *host program*.

1.7 Database Users

- Users are differentiated by the way they expect to interact with the system
- Application programmers – interact with system through DML calls
- Sophisticated users – form requests in a database query language
- Specialized users – write specialized database applications that do not fit into the traditional data processing framework
- Naïve users – invoke one of the permanent application programs that have been written previously
 - E.g. people accessing database over the web, bank tellers, clerical staff

1.8 Database Administrator

- Coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs.
- Database administrator's duties include:
 - Schema definition
 - Storage structure and access method definition
 - Schema and physical organization modification
 - Granting user authority to access the database
 - Specifying integrity constraints
 - Acting as liaison with users
 - Monitoring performance and responding to changes in requirements

1.9 Transaction Management

- A *transaction* is a collection of operations that performs a single logical function in a database application
 - E.g. **transfer funds from one account to another**
- Transaction-management component ensures that the database remains in a consistent state despite system failures

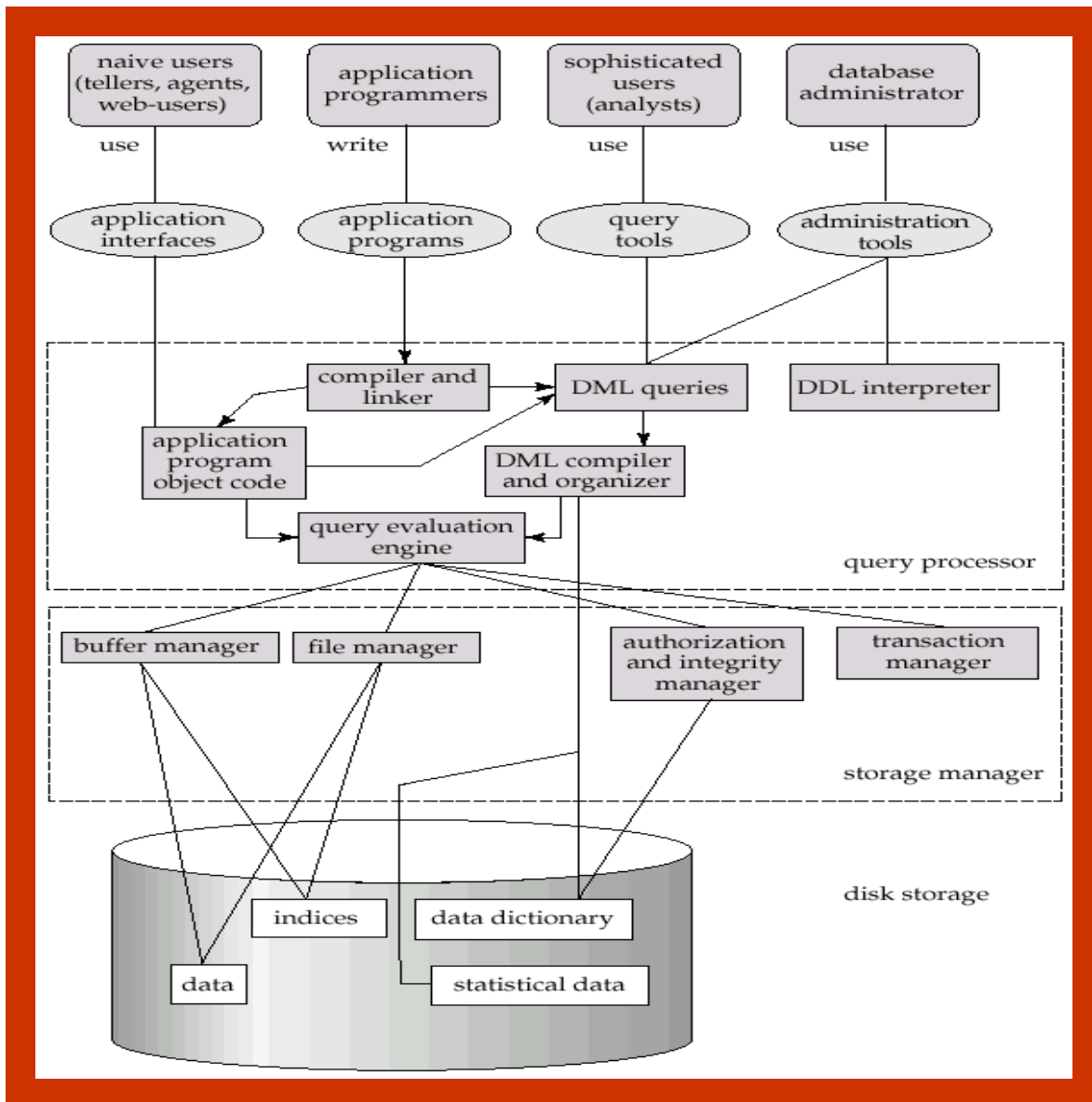
- Concurrency-control manager controls the interaction among the concurrent transactions, to ensure the consistency of the database.
 - E.g. **simultaneous withdrawals**

1.10 Storage management

- Storage manager is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
 - **interaction with the file manager**
 - **efficient storing, retrieving and updating of data**

1.11 Overall System Structure of DBMS

- The following figure shows the structure supporting parts of a DBMS with some simplification based on the relation data model.
- *A DBMS is divided into two modules (parts)*
 - Query processor
 - Storage Manager



■ Query Processor

- The Query processor components are:
 - **DDL Interpreter:** This interprets DDL statements and records the definitions in the data dictionary.
 - **DML Compiler:** as any other compiler, DML Compiler converts the DML Statements into low-level instructions.
 - **Query Evaluation:** This executes low-level instructions generated by the DML compiler.

■ Storage Manager

- A storage manager is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible for storing, retrieving and updating data in the database.
- The storage manager components include:
 - **Authorization and integrity Manager:** This tests for the satisfaction of integrity constraints and checks the authority of users to access data.

- **Transaction Manager:** This ensures that the database remains in a consistent state despite system failures, and that concurrent transaction executions proceed without conflicting.
 - **File Manager:** This manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
 - **Buffer Manager:** This is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory.
- The storage manager implements several data structures as part of the physical system implementation:
 - **Data Files:** This store the database itself.
 - **Data Dictionary:** This stores metadata about the structure of the database, in particular the schema of the database.
 - **Indices:** This provides fast access to data items that hold particular values.
- **History**
 - 1950s-60s: magnetic tape and punched cards
 - 1960s-70s: hard disks, random access, file systems
 - 1970s-80s: relational model becoming competitive
 - 1980s-90s: relational model dominant, object-oriented databases
 - 1990s-00s: web databases and XML

1.12 Entity-Relationship Model

- Entity Sets
- Relationship Sets
- Mapping Constraints
- Keys
- E-R Diagram
- Extended E-R Features
- Design of an E-R Database Schema
- Reduction of an E-R Schema to Tables

■ Entity Sets

- A *database* can be modeled as:
 - a collection of entities,
 - Relationship among entities.
- An *entity* is an object that exists and is distinguishable from other objects.
 - E.g. specific person, company, event, plant
- Entities have *attributes*
 - E.g: people have *names* and *addresses*
- An *entity set* is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, courses, books
- **Entity Sets *customer* and *loan***

321-12-3123	Jones	Main	Harrison
019-28-3746	Smith	North	Rye
677-89-9011	Hayes	Main	Harrison
555-55-5555	Jackson	Dupont	Woodside
244-66-8800	Curry	North	Rye
963-96-3963	Williams	Nassau	Princeton
335-57-7991	Adams	Spring	Pittsfield

customer

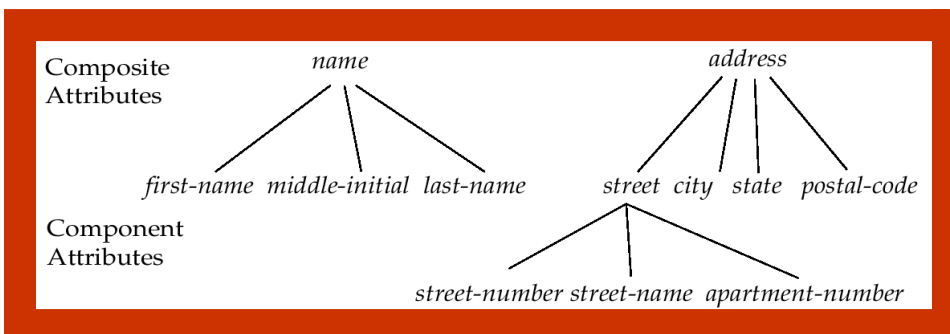
L-17	1000
L-23	2000
L-15	1500
L-14	1500
L-19	500
L-11	900
L-16	1300

loan

- **Attributes**

- An entity is represented by a set of attributes that is descriptive properties possessed by all members of an entity set.
- *Domain* – the set of permitted values for each attribute
- Attribute types:
 - **Simple and composite attributes**
 - **Single-valued and multi-valued attributes**
 - E.g. multivalued attribute: *phone-numbers*
 - **Derived attributes**
 - Can be computed from other attributes
 - E.g. *age*, given date of birth

- **Composite Attributes**



- **Relationship Sets**

- A relationship is an association among several entities

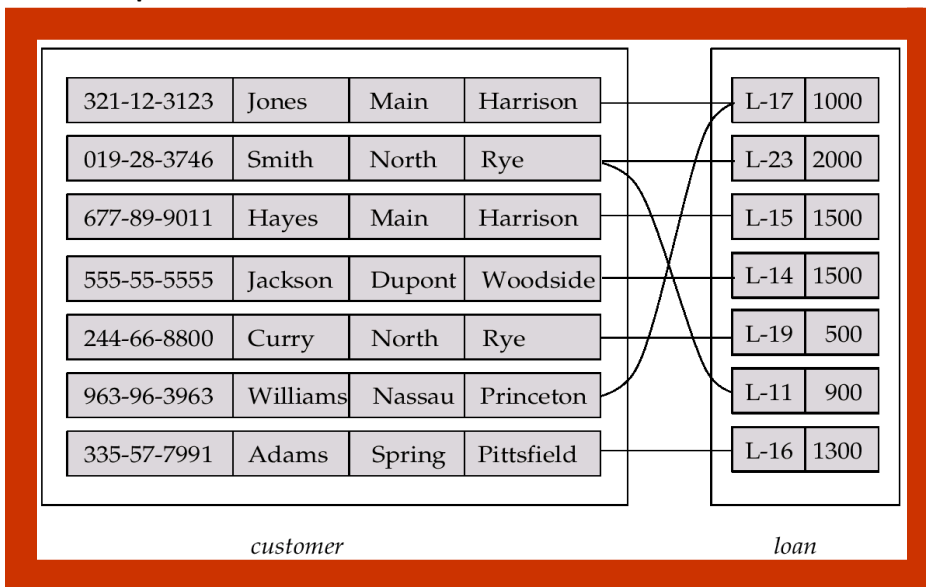
- Example:
Hayes depositor A-102
customer entity relationship set *account* entity

- A *relationship set* is a relation over $n \geq 2$ entity sets E_i :
 $\{(e_1, e_2, \dots, e_n) \mid e_i \in E_i\}$

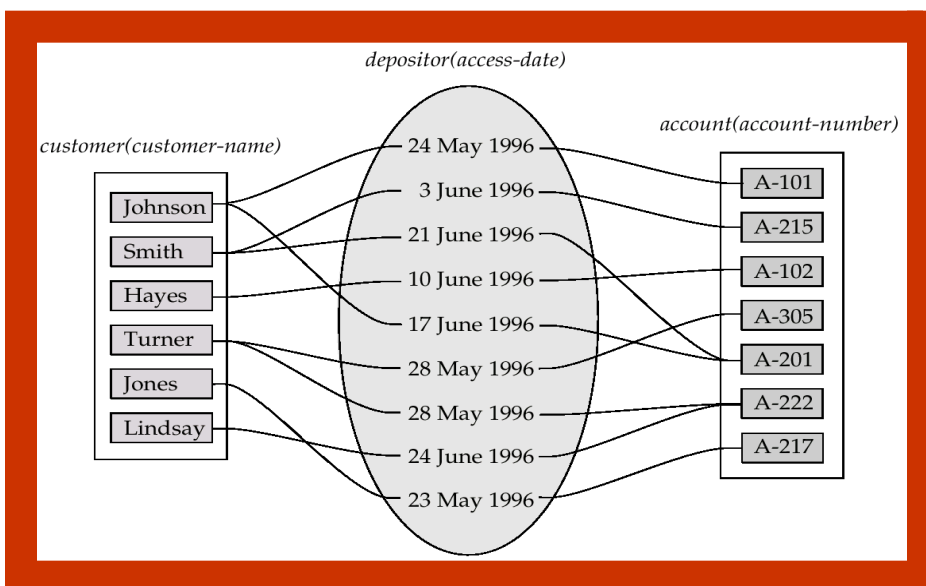
Example:

$(\text{Hayes}, \text{A-102}) \in \text{depositor}$

- **Relationship Set *borrower***



- An *attribute* can also be property of a relationship set.
- For instance, the *depositor* relationship set between entity sets *customer* and *account* may have the attribute *access-date*

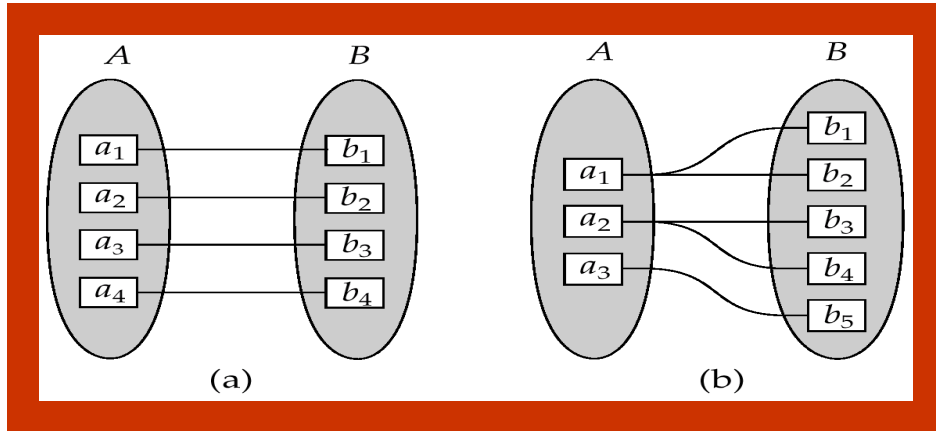


- **Degree of a Relationship Set**

- The number of entity sets that participate in a relationship set
- Relationship sets that involve two entity sets are *binary* (or degree two). Generally, most relationship sets in a database system are binary.
- Relationship sets may involve more than two entity sets.
- Relationships between more than two entity sets are rare. Most relationships are binary.

- **Mapping Cardinalities**

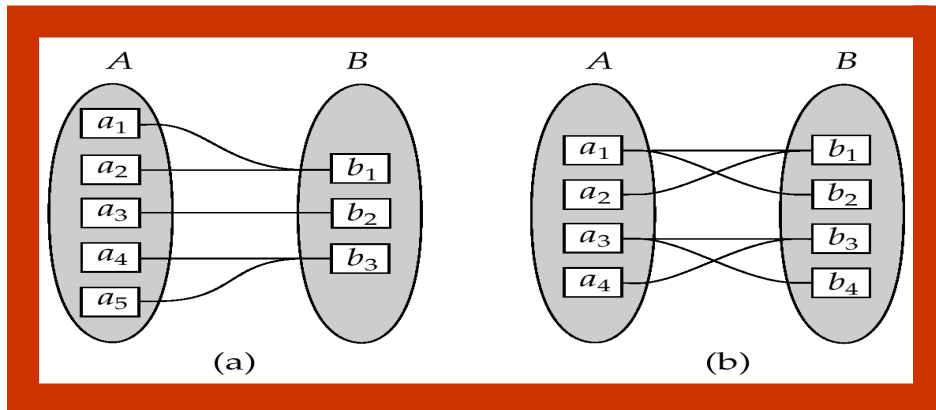
- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many



One to one

One to many

◆ Note: Some elements in A and B may not be mapped to any elements in the other set

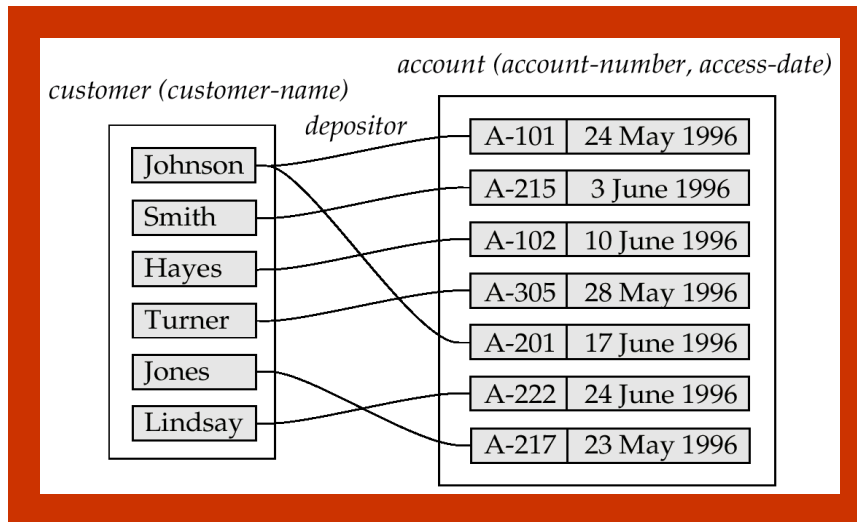


Many to one

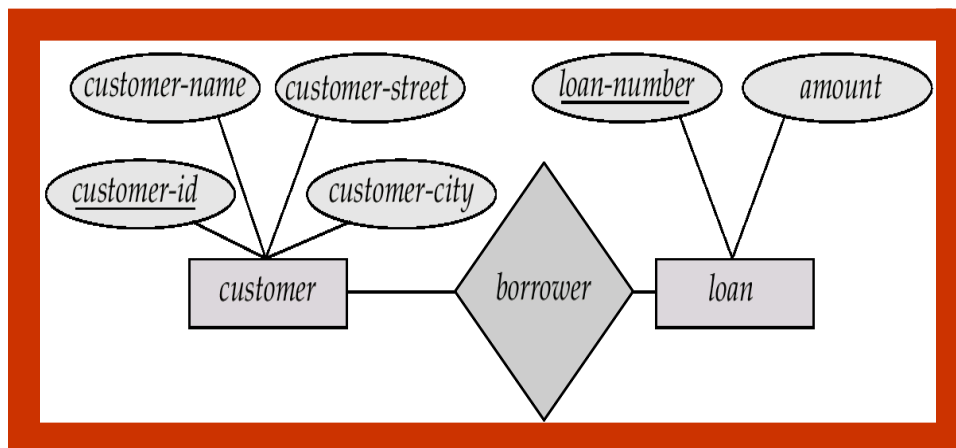
Many to many

◆ Note: Some elements in A and B may not be mapped to any elements in the other set.

- **Mapping Cardinalities affect ER Design**
- Can make *access-date* an attribute of account, instead of a relationship attribute, if each account can have only one customer
 - I.e., the relationship from account to customer is many to one,



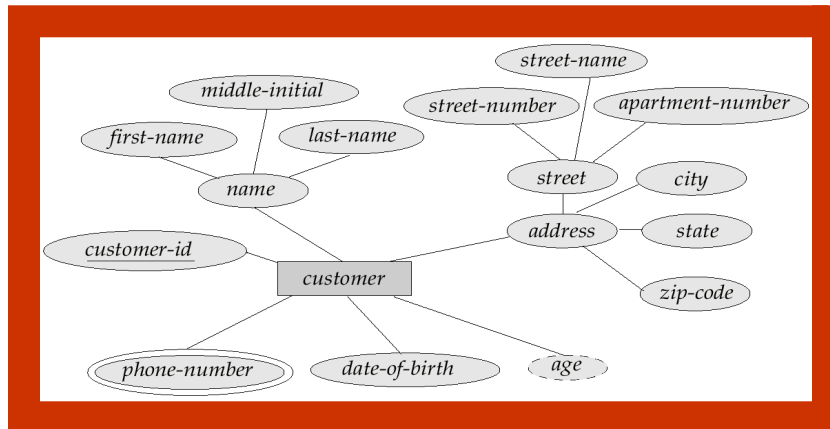
- **E-R Diagrams**



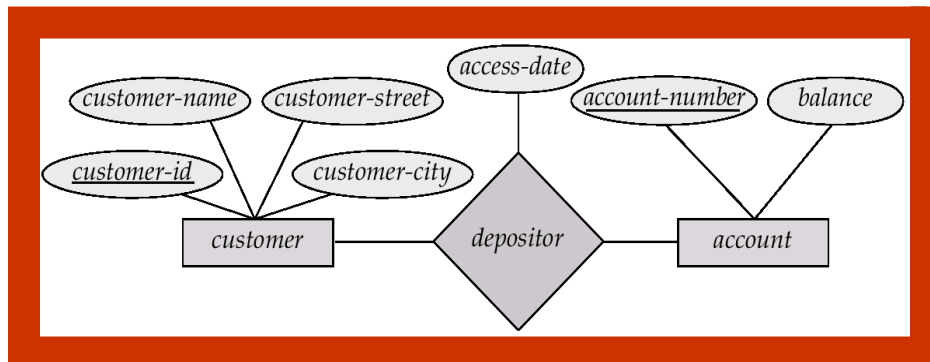
- **Rectangles** represent entity sets.
- **Diamonds** represent relationship sets.
- **Lines** link attributes to entity sets and entity sets to relationship sets.
- **Ellipses** represent attributes
 - **Double ellipses** represent multivalued attributes.
 - **Dashed ellipses** denote derived attributes.
- **Underline** indicates primary key attributes

- **E-R Diagram with Composite, Multivalued, and Derived Attributes**

- **Composite attributes:** The attributes that can be divided into subparts are known as composite attributes. Ex: name can be divided into first name, middle name and last name.
- **Multivalued attributes:** The attributes that have many values for a particular entity. Ex: name. There can be more than one name for customer.
- **Derived attribute:** The value for this type of attribute can be derived from the values of other related attributes or entities.

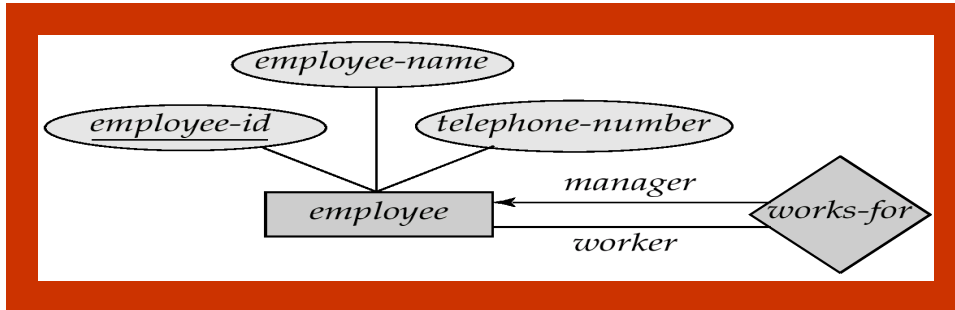


- **Relationship Sets with Attributes**



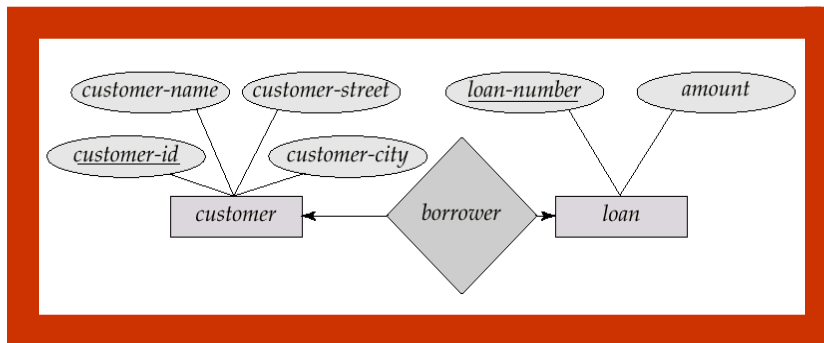
- **Roles**

- Entity sets of a relationship need not be distinct
- The labels "manager" and "worker" are called roles; they specify how employee entities interact via the works-for relationship set.
- Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.
- Role labels are optional, and are used to clarify semantics of the relationship.



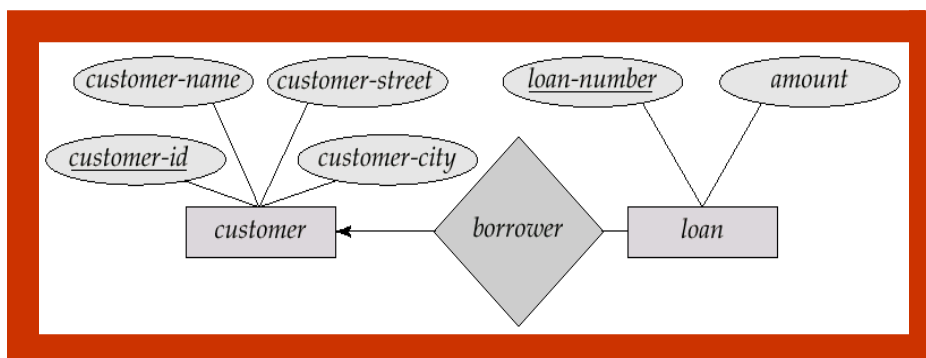
■ Cardinality Constraints

- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line ($-$), signifying “many,” between the relationship set and the entity set.
- E.g.: **One-to-one relationship:**
 - A customer is associated with at most one loan via the relationship *borrower*
 - A loan is associated with at most one customer via *borrower*



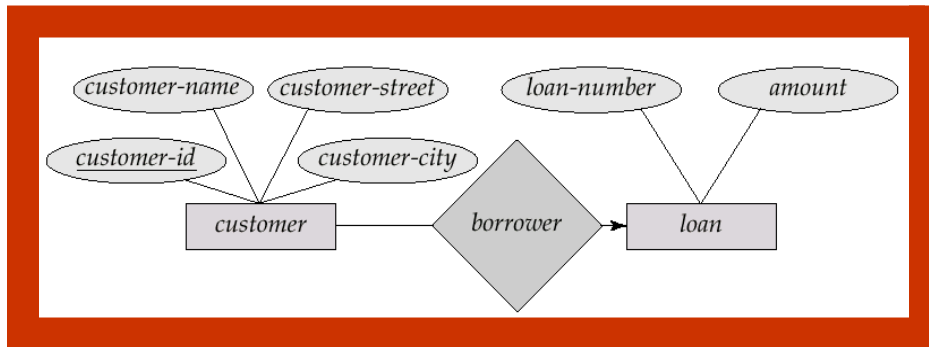
◆ One-To-Many Relationship

- In the one-to-many relationship a loan is associated with at most one customer via *borrower*, a customer is associated with several (including 0) loans via *borrower*

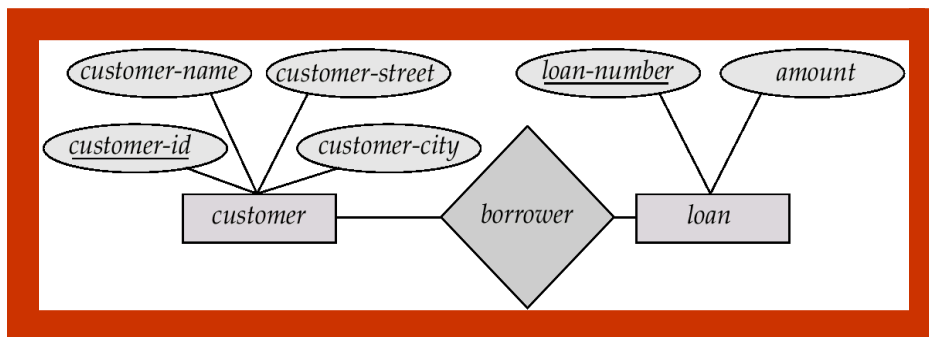


◆ Many to One Relationship

- In a many-to-one relationship a loan is associated with several (including 0) customers via *borrower*, a customer is associated with at most one loan via *borrower*



◆ Many to Many Relationship

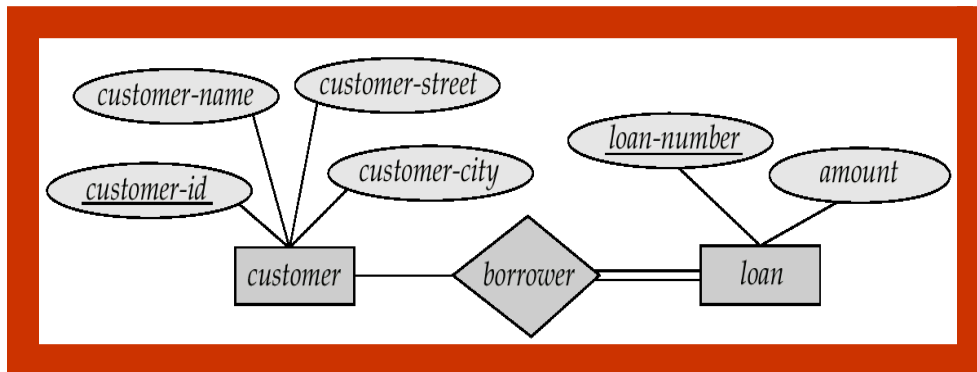


- A customer is associated with several (possibly 0) loans via *borrower*
- A loan is associated with several (possibly 0) customers via *borrower*

◆ Participation of an Entity Set in a Relationship Set

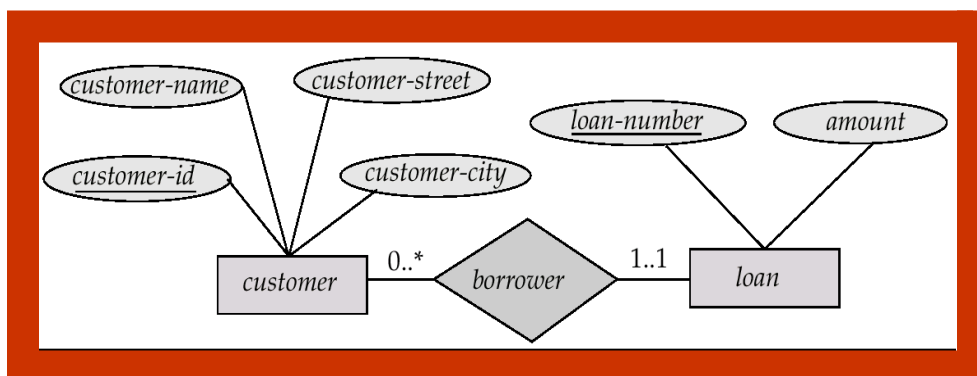
- Total participation (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set
 - E.g. participation of *loan* in *borrower* is total
 - Every loan must have a customer associated to it via *borrower*.
- Partial participation: Some entities may not participate in any relationship in the relationship set.

- E.g. participation of *customer* in *borrower* is partial



◆ Alternative Notation for Cardinality Limits

- Cardinality limits can also express participation constraints



■ Keys

- A *super key* of an entity set is a set of one or more attributes whose values uniquely determine each entity.
- A *candidate key* of an entity set is a minimal super key
 - ***Customer-id* is candidate key of *customer***
 - ***account-number* is candidate key of *account***
- Although several candidate keys may exist, one of the candidate keys is selected to be the *primary key*.

◆ Keys for Relationship Sets

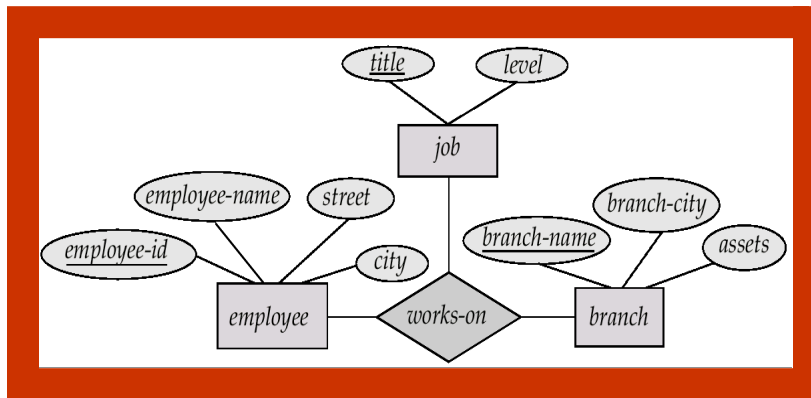
- The combination of primary keys of the participating entity sets forms a super key of a relationship set.
 - ***(customer-id, account-number)* is the super key of *depositor***

- **NOTE: this means a pair of entity sets can have at most one relationship in a particular relationship set.**

- E.g. if we wish to track all access-dates to each account by each customer, we cannot assume a relationship for each access. We can use a multivalued attribute though

- Must consider the mapping cardinality of the relationship set when deciding the what are the candidate keys
- Need to consider semantics of relationship set in selecting the *primary key* in case of more than one candidate key

- **E-R Diagram with a Ternary Relationship**



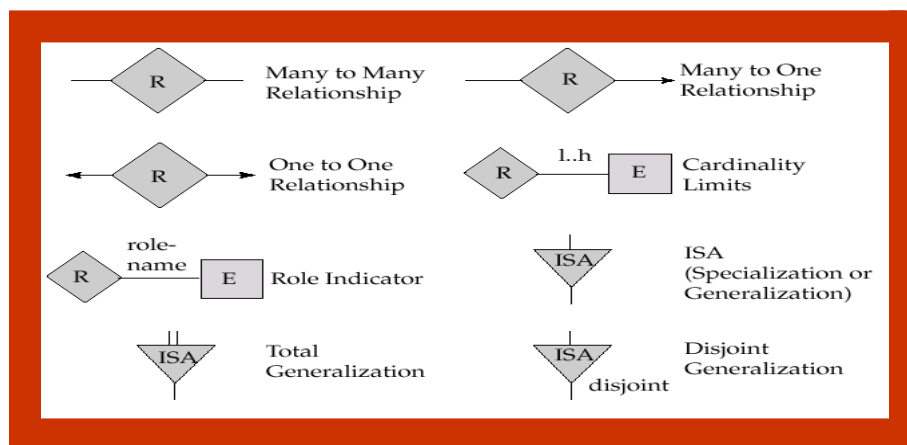
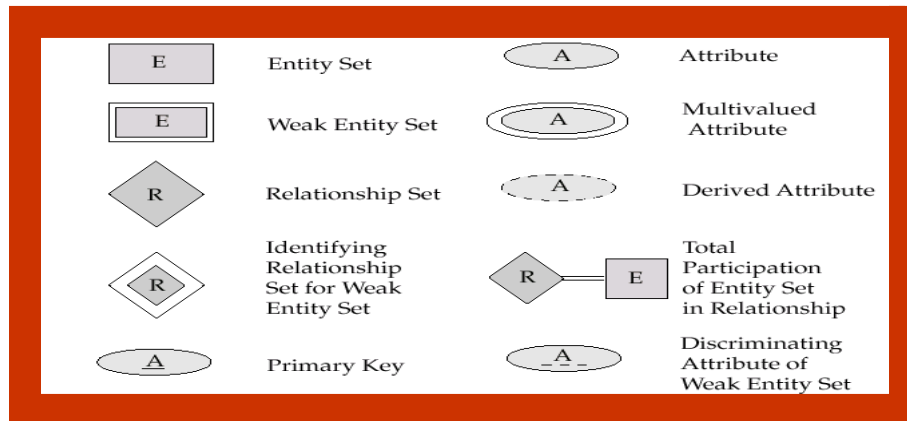
- **Cardinality Constraints on Ternary Relationship**

- We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint
- E.g. an arrow from *works-on* to *job* indicates each employee works on at most one job at any branch.
- If there is more than one arrow, there are two ways of defining the meaning.
 - E.g a ternary relationship *R* between *A*, *B* and *C* with arrows to *B* and *C* could mean
 - 1. each *A* entity is associated with a unique entity from *B* and *C* or
 - 2. each pair of entities from (*A*, *B*) is associated with a unique *C* entity, and each pair (*A*, *C*) is associated with a unique *B*
 - Each alternative has been used in different formalisms
 - To avoid confusion we outlaw more than one arrow

- **Design Issues**

- Use of entity sets vs. attributes : Choice mainly depends on the structure of the enterprise being modeled, and on the semantics associated with the attribute in question.
- Use of entity sets vs. relationship sets: Possible guideline is to designate a relationship set to describe an action that occurs between entities.
- Binary versus n-ary relationship sets : Although it is possible to replace any nonbinary (*n*-ary, for $n > 2$) relationship set by a number of distinct binary relationship sets, a *n*-ary relationship set shows more clearly that several entities participate in a single relationship.
- Placement of relationship attributes

- ◆ **Summary of Symbols Used in E-R Notation**



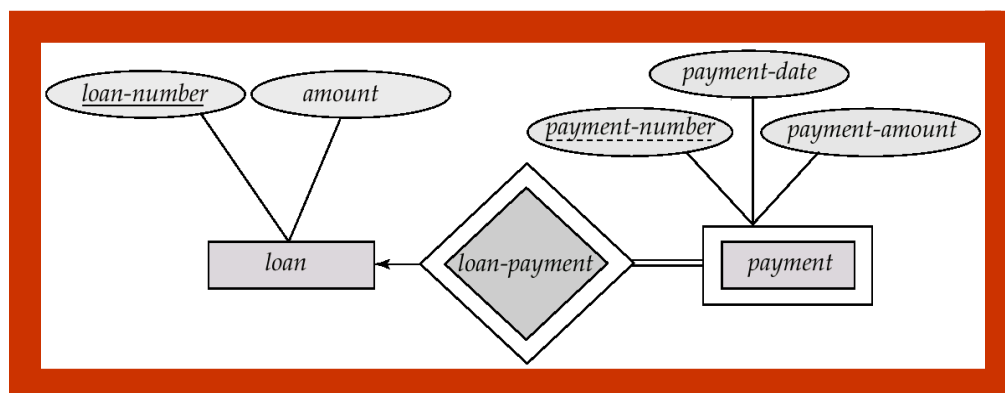
■ Extended E-R Features

- Weak entity sets
- Specialization
- Generalization
- Aggregation

◆ Weak Entity Sets

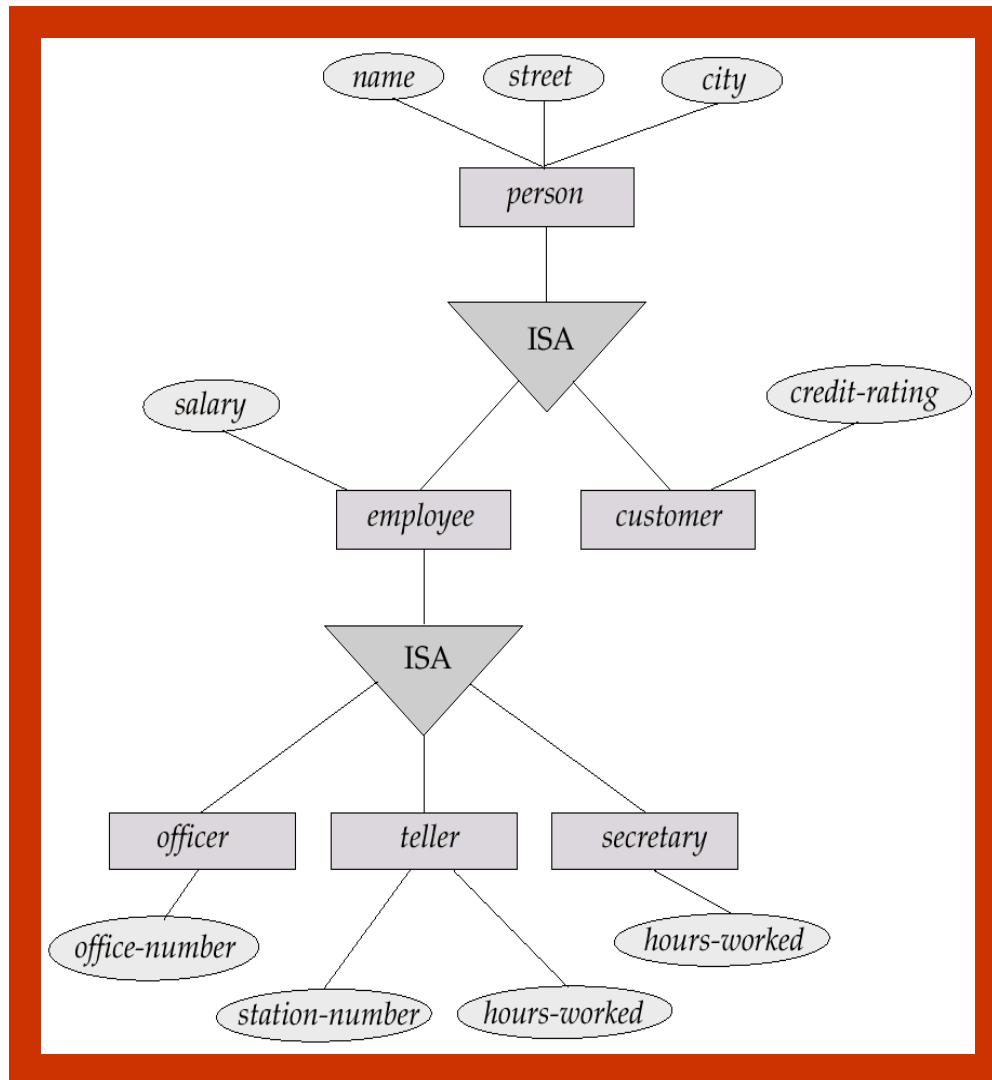
- Assumption: entity sets always have a key
 - ***This is not always true***
- Examples:
 - ***Dependents covered by an employee's insurance policy***
 - ***Film crews working at a movie studio***
 - ***Species within a genus***
- Properties
 - ***Weak entity set lacks a key***
 - ***Existence of weak entities depends on existence of corresponding entities in the "identifying entity set"***

- i.e. the participation of the weak entity in the database is only by virtue of its relationship to the identifying entity
- E.g. we're not interested in film crews except insofar as they are associated with a movie studio (an idiosyncratic property of our enterprise).
- Definition: An entity set that does not have a primary key
- The existence of a weak entity set depends on
 - **the existence of a identifying entity set**
 - **must relate to the identifying entity set via a total, many-to-one relationship set**
 - **Identifying relationship depicted using a double diamond**
- The *discriminator* of a weak entity set:
 - **the set of attributes that distinguish among all the entities of a weak entity set (also known as a partial key)**
- The primary key of a weak entity set:
 - **the primary key of the strong entity set on which the weak entity set is existence dependent, plus**
 - **the weak entity set's discriminator.**
- We depict a weak entity set by double rectangles.
- We underline the discriminator of a weak entity set with a dashed line.
- *payment-number* – discriminator of the *payment* entity set
- Primary key for *payment* – (*loan-number*, *payment-number*)



- Note: the primary key of the strong entity set is not explicitly stored with the weak entity set, since it is implicit in the identifying relationship.
- If *loan-number* were explicitly stored, *payment* could be made a strong entity, but then the relationship between *payment* and *loan* would be duplicated by an implicit relationship defined by the attribute *loan-number* common to *payment* and *loan*.
- **More Weak Entity Set Examples**

- In a university, a *course* is a strong entity and a *course-offering* can be modeled as a weak entity
 - The discriminator of *course-offering* would be *semester* and *year*
- If we model *course-offering* as a strong entity we would model *course-number* as an attribute.
 - Then the relationship with *course* would be implicit in the *course-number* attribute
- **Specialization**
- Top-down design process
 - Start with few entity sets having many attributes
 - E.g. person entity may have attributes suitable for students, lecturers, employees, employers, etc.
- we identify distinctive subgroupings within an entity set
- These subgroupings become lower-level entity sets
 - They have attributes or participate in relationships that do not apply to the higher-level entity set
- Depicted by a *triangle* component labeled ISA
- E.g. *customer* “is a” *person*
- **Inheritance**
 - a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.
- **Specialization Example**



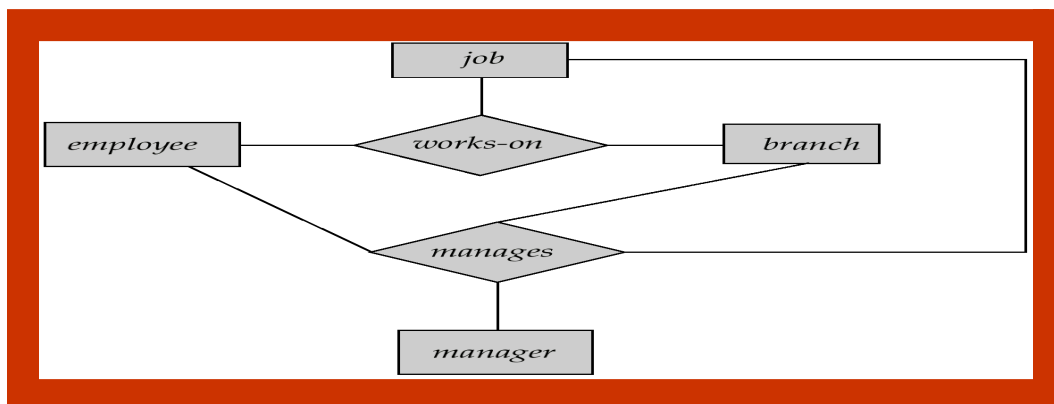
■ Generalization

- A bottom-up design process
 - start with lots of distinct entities that share attributes
 - Combine a number of entity sets that share the same attributes into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.

◆ Specialization and Generalization

- Can have multiple specializations of an entity set based on different features.
- E.g. *permanent-employee* vs. *temporary-employee*, in addition to *officer* vs. *secretary* vs. *teller*.
- Each particular employee would be
 - a member of one of *permanent-employee* or *temporary-employee*,
 - and also a member of one of *officer*, *secretary*, or *teller*

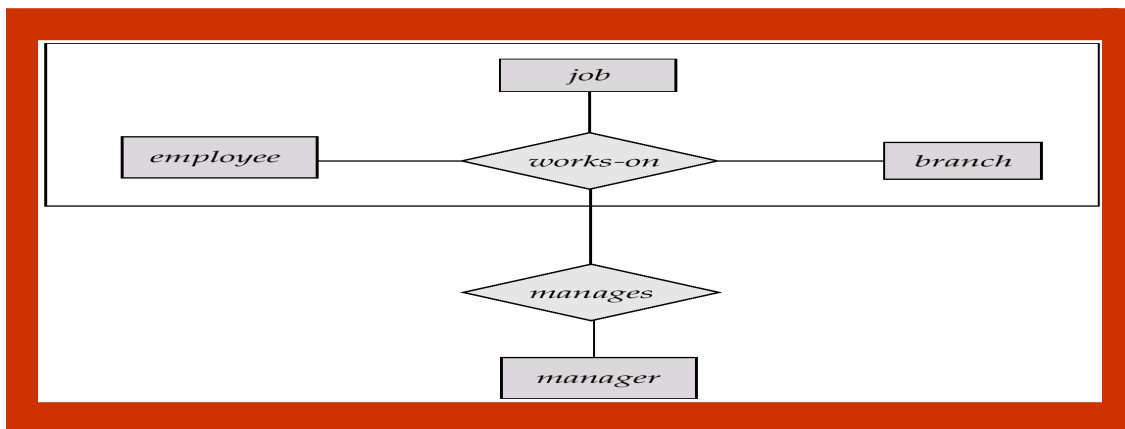
- The ISA relationship also referred to as **superclass - subclass** relationship.
- ◆ **Design Constraints on a Specialization/Generalization**
- Constraint on which entities can be members of a given lower-level entity set.
 - **condition-defined**
 - E.g. all customers over 65 years are members of *senior-citizen* entity set; *senior-citizen* ISA *person*.
 - **user-defined**
- Constraint on whether or not entities may belong to more than one lower-level entity set within a single generalization.
 - **Disjoint**
 - an entity can belong to only one lower-level entity set
 - write *disjoint* next to the ISA triangle
 - **Overlapping**
 - an entity can belong to more than one lower-level entity set
- Completeness constraint
 - **Does an entity in the higher-level entity set have to belong to at least one of the lower-level entity sets?**
- Total
 - an entity must belong to one of the lower-level entity sets
- Partial
 - an entity need not belong to one of the lower-level entity sets
- **Aggregation**
- Consider the ternary relationship works-on
- Suppose we want to record managers for tasks performed by an employee at a branch.



- *works-on* and *manages* represent overlapping information
 - **Every *manages* relationship corresponds to a *works-on* relationship**

- some *works-on* relationships may not correspond to any *manages* relationships
- we can't discard the *works-on* relationship
- Eliminate this redundancy via *aggregation*
 - Treat *works-on* relationship as an abstract entity
 - Allow relationships between relationships!
- Abstraction of relationship into new entity
- Without introducing redundancy, the following diagram represents:
 - An employee works on a particular job at a particular branch
 - An employee, branch, job combination may have an associated manager

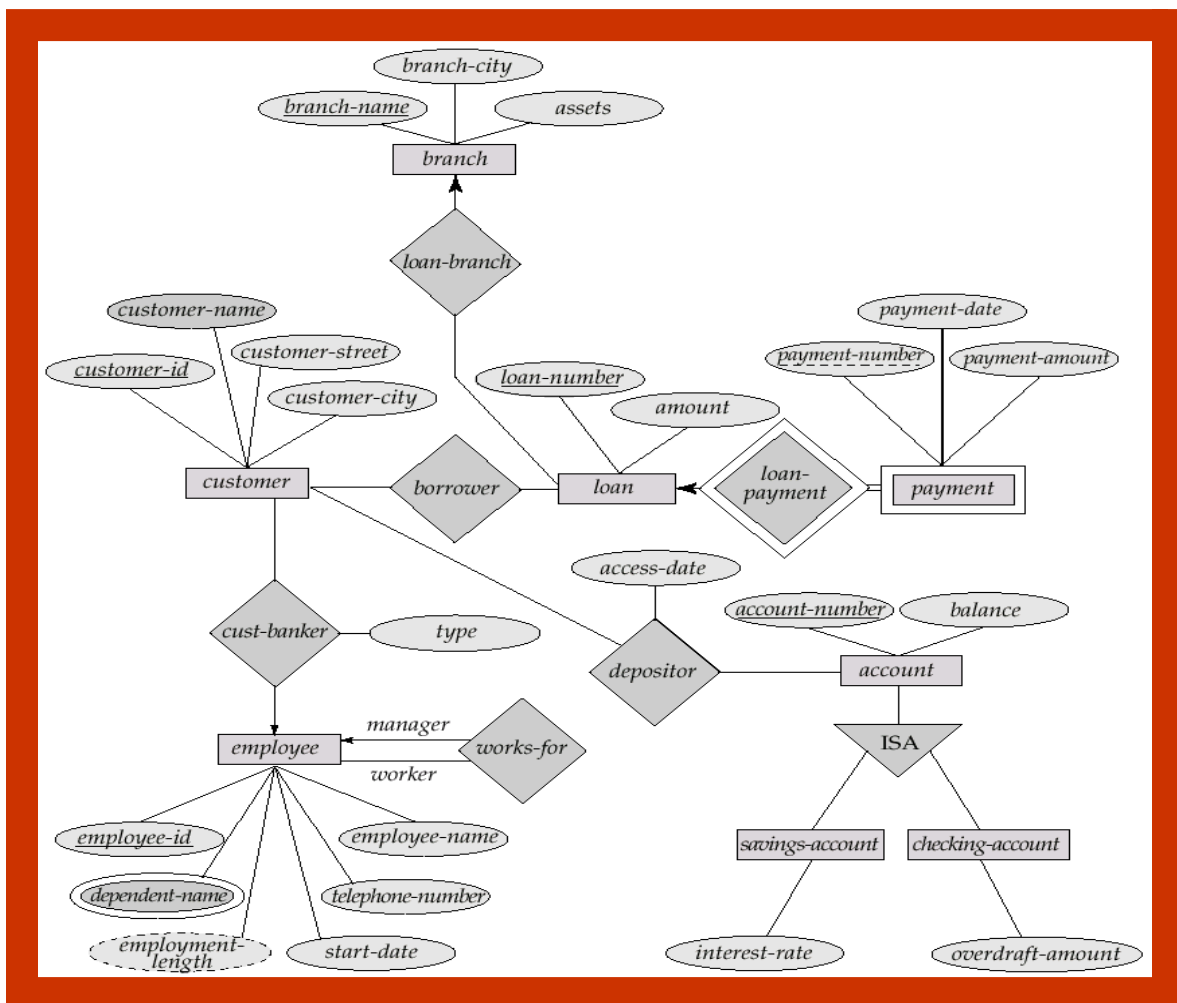
▪ **E-R Diagram with Aggregation**



■ **E-R Design Principles**

- Faithfulness
 - Entities, attributes and relationships should reflect reality
 - Sometimes the correct approach is not obvious
 - E.g. course and instructor entities and teaching relationship
 - What are the cardinality constraints? It depends...
- Avoiding Redundancy
 - No information should be repeated
 - Wastes space, leads to consistency problems
- Simplicity
 - Some relationships may be unnecessary
 - E.g. student member-of student-body attends course vs student attends course

- Choosing the right kind of element
 - The use of an attribute or entity set to represent an object
 - Whether a real-world concept is best expressed by an entity set or a relationship set
 - Choosing the right relationships
 - The use of a ternary relationship versus a pair of binary relationships
 - The use of a strong or weak entity set.
 - The use of specialization/generalization – contributes to modularity in the design.
 - The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.
- E-R Diagram for a Banking Enterprise



■ Reduction of an E-R Schema to Tables

- Primary keys allow entity sets and relationship sets to be expressed uniformly as *tables* which represent the contents of the database.

- A database which conforms to an E-R diagram can be represented by a collection of tables.
- For each entity set and relationship set there is a unique table which is assigned the name of the corresponding entity set or relationship set.
- Each table has a number of columns (generally corresponding to attributes), which have unique names.
- Converting an E-R diagram to a table format is the basis for deriving a relational database design from an E-R diagram.

▪ Representing Entity Sets as Tables

- A strong entity set reduces to a table with the same attributes.

<i>customer-id</i>	<i>customer-name</i>	<i>customer-street</i>	<i>customer-city</i>
019-28-3746	Smith	North	Rye
182-73-6091	Turner	Putnam	Stamford
192-83-7465	Johnson	Alma	Palo Alto
244-66-8800	Curry	North	Rye
321-12-3123	Jones	Main	Harrison
335-57-7991	Adams	Spring	Pittsfield
336-66-9999	Lindsay	Park	Pittsfield
677-89-9011	Hayes	Main	Harrison
963-96-3963	Williams	Nassau	Princeton

◆ Composite and Multivalued Attributes

- Composite attributes are flattened out by creating a separate attribute for each component attribute
 - E.g. given entity set *customer* with composite attribute *name* with component attributes *first-name* and *last-name* the table corresponding to the entity set has two attributes
name.first-name and *name.last-name*
- A multivalued attribute M of an entity E is represented by a separate table EM
 - Table EM has attributes corresponding to the primary key of E and an attribute corresponding to multivalued attribute M
 - E.g. Multivalued attribute *dependent-names* of *employee* is represented by a table *employee-dependent-names(employee-id, dname)*
 - Each value of the multivalued attribute maps to a separate row of the table EM
 - E.g., an employee entity with primary key John and dependents Johnson and Johndotir maps to two rows:
(John, Johnson) and (John, Johndotir)

◆ **Representing Weak Entity Sets**

- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set

<i>loan-number</i>	<i>payment-number</i>	<i>payment-date</i>	<i>payment-amount</i>
L-11	53	7 June 2001	125
L-14	69	28 May 2001	500
L-15	22	23 May 2001	300
L-16	58	18 June 2001	135
L-17	5	10 May 2001	50
L-17	6	7 June 2001	50
L-17	7	17 June 2001	100
L-23	11	17 May 2001	75
L-93	103	3 June 2001	900
L-93	104	13 June 2001	200

◆ **Representing Relationship Sets as**

- A many-to-many relationship set is represented as a table with columns for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- E.g.: table for relationship set *borrower*

<i>customer-id</i>	<i>loan-number</i>
019-28-3746	L-11
019-28-3746	L-23
244-66-8800	L-93
321-12-3123	L-17
335-57-7991	L-16
555-55-5555	L-14
677-89-9011	L-15
963-96-3963	L-17

1.13 Conceptual Design with ER Model

- Developing an ER diagram presents several design issues, including the following:
 - Entity versus Attribute.
 - Entity versus Relationship
 - Binary versus Ternary Relationships.
 - Aggregation versus Ternary Relationships.
- **Entity versus Attribute**

- While identifying the attributes of an entity set, it is sometimes not clear, whether a property should be modeled as an attribute or as an entity set.
- Example: consider the entity set employee with attributes employees name and telephone number. It can easily be said that a telephone is an entity in its own right with attributes telephone number and location. If we take this point of views, the employee entity set must be redefined as follows:
 - The employee entity set with attribute employee name.
 - The telephone entity set with attributes telephone number and location.
 - The relationship set employee telephone, which denotes the association between employees and the telephones that they have.
- The main difference between these two definitions of an employee is as follows:
 - In the first case, the definition implies that every employee has one telephone number associated with him.
 - In the second case, the definition implies that all employees may have several telephone number associated with them.
- Thus, the second definition is more general than the first one, and may more accurately reflect the real world situation. Even if we are given that each employee has only one telephone number associated with him, the second definition may still be more appropriate because the telephone is shared among several employees.
- However, it is appropriate to have employees-name as an attribute of the employee entity set instead of an entity because most of the employees have single name.

■ Entity versus Relationship

- It is not always clear whether an object is best expressed by an entity set or a relationship set.
- Example: assume that, a bank loan is modeled as an entity. An alternative is to model a loan not as an entity, but rather as a relationship between customers and branches, with loan number and amount as descriptive attributes. Each loan is represented by a relationship between a customer and a branch.
- If every loan is held by exactly one customer and customer is associated with exactly one branch, we may find satisfactory the design, where a loan is represented as a relationship. But, with this design, we cannot represent conveniently a situated in which several customers hold a loan jointly. We must define a separate relationship for each holder of the joint loan. Then, we must replicate the values for the descriptive attributes loan-number and amount in each such relationship. Each such relationship must of course, have the same value for the descriptive attributes loan number and amount.
- Two problems arise as a result of the replication:
 - The data are stored multiple times, wasting storage space.

- Updates leave the data in an inconsistent state.
- One possible guideline is determining whether to use an entity set or a relationship set to designate a relationship set, an action that occurs between entities. This approach can also be useful in deciding whether certain attributes may be more appropriately expressed as relationships.

■ Binary versus ternary Relationships

- It is always possible to replace a non-binary (n-ary, for $n > 2$) relationship set by a number of distinct binary relationship sets.
- Example: for simplicity, consider the abstract ternary ($n=3$) relationship set R , relating entity sets A , B , C . We replace the relationship set R by an entity set E , and create three relationship sets:
 - * R_A , relating E and A
 - * R_B , relating E and B
 - * R_C , relating E and C
- If the relationship set R has any attributes, these are assigned to entity set E ; otherwise, a special identifying attribute is created for E . For each relationship (a_i, b_i, c_i) in the relationship set R , we create a new entity e_i in the entity set E .
- Then, in each of the three new relationship sets, we insert a relationship as follows:
 - * (e_i, a_i) in R_A
 - * (e_i, b_i) in R_B
 - * (e_i, c_i) in R_C
- We can generalize this process in a straight forward manner to n-ary relationship sets. Thus, conceptually, we can restrict the E-R model to include only binary relationship sets.

■ Aggregation versus Ternary Relationships

- The choice between using aggregation or a ternary relationship is mainly determined by the existence of a relationship that relates a relationship set to an entity set. The choice may also be guided by certain integrity constraints that we want to express.
- Example: consider the constraint that each sponsorship be monitored by at most one employee. We can express this constraint in terms of the sponsor's relationship set. On the other hand, we can easily express the constraint by drawing an arrow from the aggregated relationship sponsors to the relationship Monitors. Thus, the presence of such a constraint serves as another reason for using aggregation rather than a ternary relationship set.

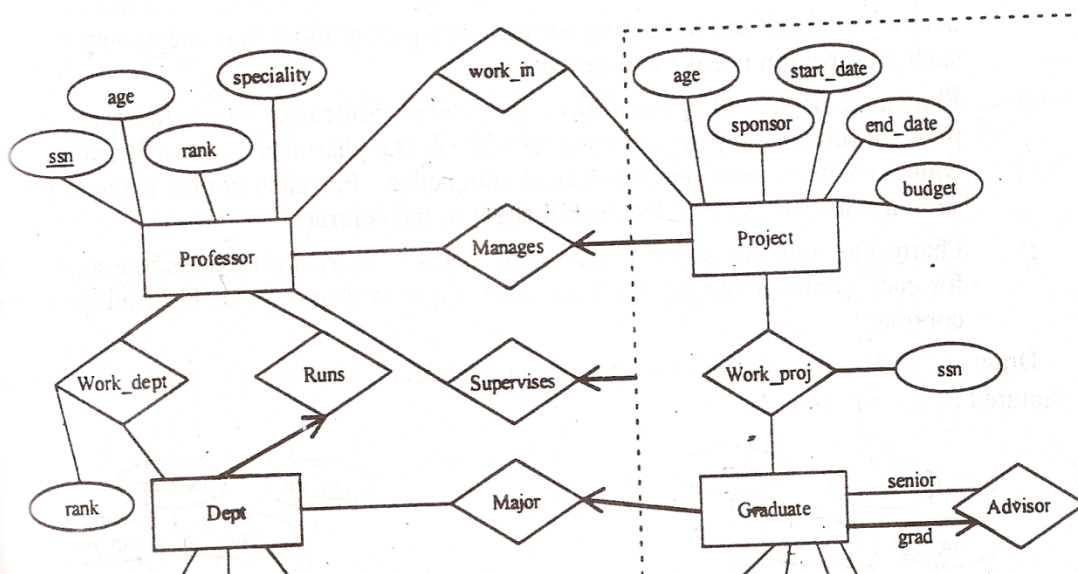
1.14 Conceptual Design for Large Database

- Designing database for large organization takes efforts of more than a single designer. It diagrammatically represents the complete database and enables the user who provides inputs to database, to understand the complete functionality of database.
- Large databases are modeled in two methodologies.
 - The requirements of all the users are collected. The conflicting requirements are resolved and a final conceptual view is generated to satisfy the requirements of all users.
 - In the other method, the user provides his requirements; the designer generates a conceptual view for the requirements. Like wise all the conceptual views from all user requirements are generated and a comprehensive conceptual view that satisfies all the requirements is generated.

1.15 E-R Diagrams Examples

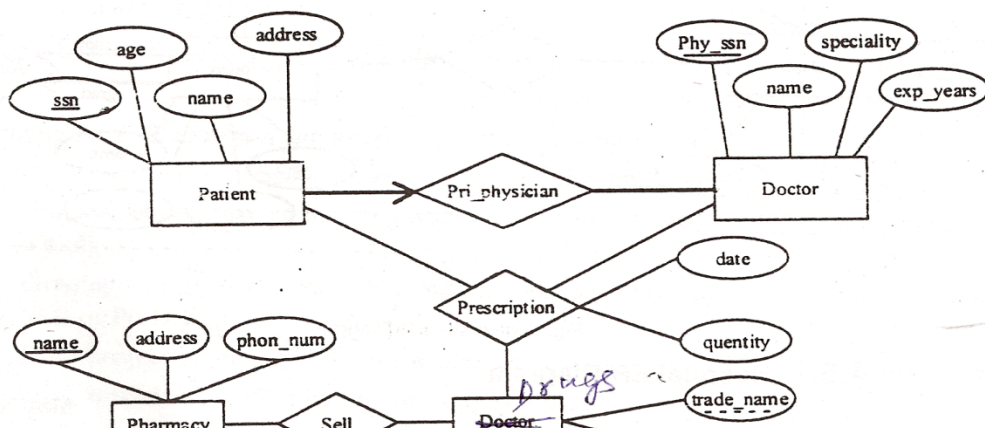
■ **University E-R Diagram: Construct an E-R diagram for university database with following information:**

- Professors have an SSN, a name, an age, a rank, and a research speciality.
- Projects have a project number, a sponsor name, a starting date, an ending data and a budget.
- Graduate students have an SSN, a name, an age, and a degree program (ex: M.S or Ph.D).
- Each project is managed by one professor.
- Each project is worked by one or more professors.
- Professors can manage and /or work on multiple projects.
- Each project is worked by one or more graduate students (known as the projects research assistants).
- When graduate students work on a project, a professor must supervise their work on the project. Graduate students can work on multiple projects, in this case they can have a different supervisor for each one.
- Departments have a department number, a department name and a main office.
- Departments have a professor who runs the department.
- Professors work in one or more departments, and for each department that they work in a time percentage is associated with their job.
- Graduate students have a major department in which they are working on their degree.
- Each graduate student has another, more senior graduate student (known as student advisor) who advises him or her on what courses to take.

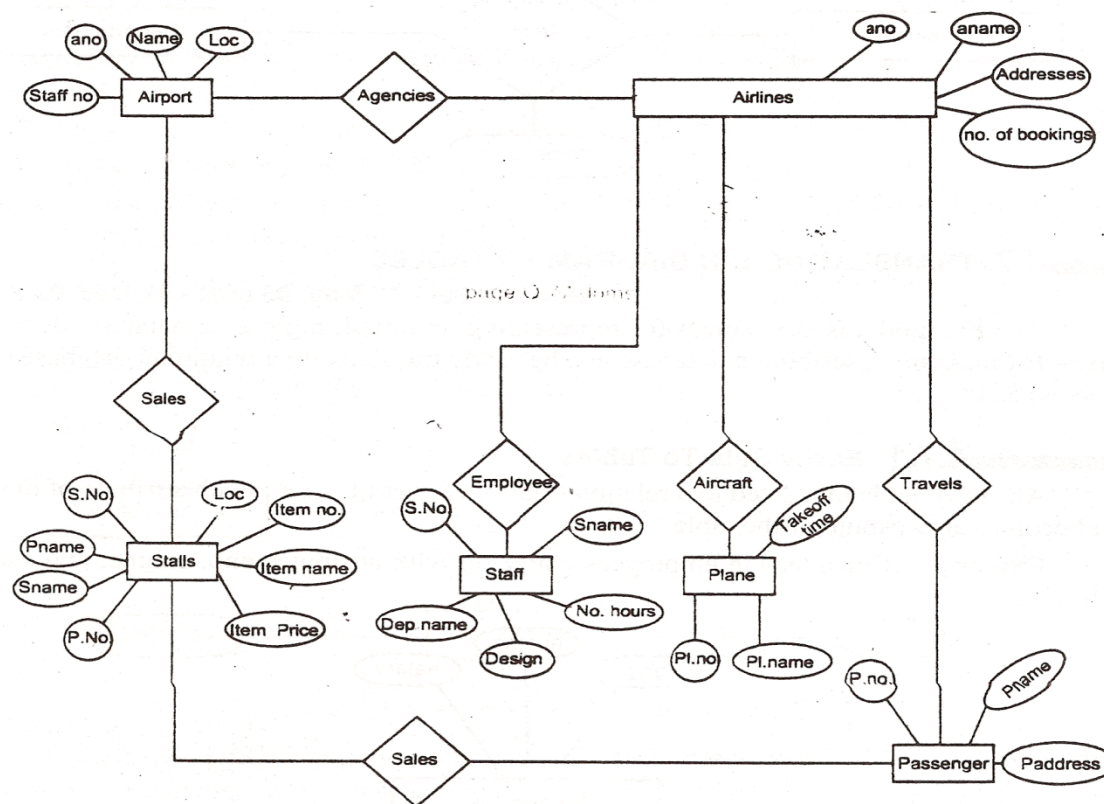


■ **Hospital ER Diagram: Construct an E-R diagram for a hospital database with the following information:**

- Patients are identified by an SSN, and their names, addresses, and ages must be recorded.
- Doctors are identified by an SSN. For each doctor, the name, speciality and years of experience must be recorded.
- Each pharmaceutical company is identified by name and has a phone number.
- For each drug, the trade name and formula must be recorded. Each drug is sold by a give Pharmaceutical Company, and the trade name identifies a drug uniquely from among the products of that company. If a pharmaceutical company is deleted, you need not keep track of its products any longer.
- Each pharmacy has a name, address and phone number.
- Every patient has a primary physician. Every doctor has at least one patient.
- Each pharmacy sells several drugs and has a price for each. A drug could be sold at several pharmacies, and the price could vary from one pharmacy to another.
- Doctors prescribe drugs for patients. A doctor could prescribe one or more drugs for several patients, and a patient could obtain prescriptions from several doctors. Each prescription has a date and a quantity associated with it. You can assume that, if a doctor prescribes the same drug for the same patient more than one, only the last such prescription needs to be stored.
- Pharmaceutical Companies have long-term contacts with pharmacies. A pharmaceutical company can contract with several pharmacies, and a pharmacy can contract with several pharmaceutical companies. For each contract, you have to store a start date, an end date, and the text of the contract.
- Pharmacies appoint a supervisor for each contract. There must always be a supervisor for each contract, but the contract supervisor can change over the lifetime of the contract.



- **Airline Reservation System ER Diagram:** Explain with ER diagram an AIRLINE RESERVATION SYSTEM identify the entities, attributes and relationship exist among entities.



■ **Store ER Diagram: Construct the ER diagram for the database of a store which is as follows:**

- set of employees
- Set of departments
- Items sold
- Manufacturer for the items.

